TRABAJO PRÁCTICO INDIVIDUAL DE DESARROLLO CON INTELIGENCIA ARTIFICIAL



Materia: Desarrollo de Software (DDS) - UTN FRBA - Segundo Cuatrimestre 2026
Alumno: Franco Ivan Ball Abalos
Proyecto: AI Ticket It - Plataforma de Venta de Entradas y Gestión de Espectáculos

INTRODUCCIÓN

Una vez concluido el desarrollo completo de AI Ticket It y con todas las funcionalidades, pruebas y pantallas en funcionamiento, este documento reúne mi reflexión final sobre lo que significó encarar un proyecto de software de punta a punta asistido por Inteligencia Artificial.

Esta reflexión no nace sólo del trabajo con AI Ticket It, sino también de compararlo con experiencias personales previas en otros proyectos académicos y laborales. En el pasado, desarrollar una solución full-stack con esta profundidad solía implicar largas jornadas dedicadas casi exclusivamente a tareas mecánicas: pelear con configuraciones de entornos, recordar sintaxis de librerías, armar estructuras repetitivas de controladores o postergar la escritura de tests por falta de tiempo. Al incorporar herramientas de IA en el día a día, el contraste fue evidente: la velocidad para plasmar ideas en código creció exponencialmente, pero al mismo tiempo aparecieron desafíos nuevos relacionados con el control del código generado, la toma de decisiones arquitectónicas y la necesidad de no dar por sentado nada de lo que la IA propone.

A continuación, comparto un balance sincero sobre este proceso, las herramientas que mejor funcionaron, los momentos donde la IA aceleró el desarrollo de forma sorprendente, las fricciones reales que surgieron en el camino, y los aprendizajes que me llevo para mi futuro profesional.

HERRAMIENTAS DE INTELIGENCIA ARTIFICIAL UTILIZADAS

A la hora de encarar el proyecto, decidí no atarme a una única herramienta para todo, sino dividir el trabajo en dos grandes fases complementarias: una etapa de diseño y estrategia técnica, y otra de ejecución e implementación directa en el código. Esta división me permitió aprovechar lo mejor de cada modelo según la tarea.

Para ayudarme con la toma de decisiones arquitectónicas, el diseño conceptual y el armado del plan de desarrollo, me apoyé fuertemente en Claude. Su capacidad para razonar sobre abstracciones complejas me resultó ideal para debatir cómo estructurar las capas de Clean Architecture y Domain-Driven Design. Con Claude fui discutiendo y puliendo el alcance de las entidades del dominio, las reglas de negocio para considerar la mayor cantidad de edge-case posibles, los diagramas de estados de una compra y la división de los sprints con sus respectivos criterios de aceptación. Funcionó como un verdadero consultor técnico con el cual contrastar alternativas de diseño antes de escribir la primera línea de código.

Una vez que las decisiones estructurales y el plan de sprints estuvieron claros, pasé a la fase de implementación práctica utilizando Google Antigravity como entorno principal de desarrollo. En esta oportunidad, decidí darle la mayor cantidad de contexto posible a cada agente para que pudieran producir resultados de mejor calidad. Luego de que el agente terminara de implementar los cambios, yo revisaba que estos cambios estuvieran acorde a lo que se pretendía y, en caso de ser necesario, marcaba las áreas donde se podía mejorar

la solución desarrollada. Para otorgar la posibilidad a los agentes de probar su propio código y asegurar el funcionamiento del mismo, le instruí a los agentes que corran los comandos en la terminal para ejecutar tests y levantar los servicios. Así, cada vez que yo tenía que revisar, me aseguraba de que por lo menos el código que estaba revisando no tuviera errores de tipado y me permitía concentrarme más en la calidad de la solución.

Esta combinación entre Claude para el pensamiento estratégico y Google Antigravity para la construcción e ingeniería del código resultó muy orgánica y fluida, evitando la saturación de contexto y asegurando que cada herramienta sea utilizada en el terreno donde mayor valor aportaba.

VENTAJAS Y GANANCIAS REALES DE PRODUCTIVIDAD

Al evaluar el impacto real que tuvo este flujo de trabajo, la ventaja más evidente no fue solo la rapidez, sino el cambio cualitativo en la forma de encarar el desarrollo. En proyectos anteriores, una parte desmedida de las horas de trabajo se consumía en tareas puramente mecánicas: levantar la estructura del servidor, lidiar con configuraciones iniciales, escribir controladores repetitivos con sus respectivas rutas o pelear con la sintaxis de estilos y maquetado. En AI Ticket It, delegar ese trabajo en la IA me permitió comenzar a iterar sobre la lógica del negocio casi desde el primer día. Lo que normalmente me hubiera llevado días de configuración y escritura de código repetitivo se resolvió en cuestión de horas, manteniendo un estándar limpio y consistente en todo el proyecto.

Sin embargo, donde sentí el beneficio más profundo fue en la construcción de la estrategia de pruebas. En mi experiencia, en el desarrollo de software bajo presión de entregas o plazos ajustados, el testing suele ser la primera víctima: se prueban un par de caminos felices y se posterga o saltea la cobertura exhaustiva. Con la asistencia de la IA, esa dinámica se invirtió por completo. La facilidad para plantear escenarios de prueba me permitió construir una cantidad masiva de tests unitarios y de integración en Jest y suites completas de extremo a extremo en Playwright. La IA fue fundamental no solo para escribir el código del test, sino para sugerir caminos tristes y edge cases a los que a veces uno pasa por alto.
Poder blindar la aplicación con más de cien pruebas en paralelo al desarrollo me dio mucha tranquilidad para refactorizar y avanzar sprint a sprint sin miedo a romper funcionalidades previas.

Otro punto alto fue la incorporación natural de las directrices de accesibilidad vistas en la materia. Muchas veces, agregar etiquetas descriptivas a cada botón, asegurar tamaños táctiles mínimos de 48 píxeles en móviles o configurar roles semánticos para lectores de pantalla se percibe como una carga tediosa que se deja para el final. Al trabajar en conjunto con la IA, estas prácticas se integraron de forma orgánica desde la creación del primer componente. Cada modal, tarjeta o campo de formulario nacía ya con sus accessibilityLabel y roles correspondientes. Esto no sólo elevó la calidad inclusiva de la aplicación, sino que terminó siendo el ancla perfecta para que las pruebas de Playwright funcionaran de manera

sólida y determinística, buscando elementos por su semántica y no por clases visuales frágiles.

Por último, rescato enormemente la capacidad de mantener una documentación viva a la par del código. En lugar de tener que redactar tediosos informes al final del proyecto intentando recordar qué se hizo semanas atrás, la IA facilitó registrar cada decisión de diseño, actualizar la especificación interactiva de Swagger OpenAPI y documentar la evolución de los casos de uso en tiempo real. En definitiva, la gran ventaja no fue programar en "piloto automático", sino elevar mi rol como desarrollador: pasar de pelearme con el código a desempeñarme como un verdadero diseñador de soluciones, concentrando toda mi atención en la arquitectura, la experiencia de usuario y las reglas del dominio.

FRICCIONES, DESAFÍOS Y DIFICULTADES TÉCNICAS ENCONTRADAS

A pesar de los indudables beneficios en velocidad y cobertura, el desarrollo no fue todo color de rosas. Trabajar con herramientas de inteligencia artificial implica comprender rápidamente que los modelos no poseen un entendimiento holístico del entorno de ejecución, sino que predicen soluciones que "parecen" correctas. Cuando esas sugerencias se contrastan contra un proyecto real, con dependencias específicas y entornos híbridos, surgen fricciones donde la intervención y el criterio técnico resultan determinantes.

Uno de los primeros obstáculos importantes apareció en el ecosistema de React Native y Expo. En varias oportunidades, la IA tendía a sugerir paquetes muy populares de npm para resolver componentes específicos (como selectores de fecha nativos, librerías de alertas estilizadas o utilidades de calendario). El problema era que muchas de esas librerías incluían código nativo no soportado por el flujo gestionado de Expo o, peor aún, fallaban silenciosamente al compilar para la variante web. Tuve que frenar deliberadamente esas propuestas y orientar a la herramienta para que utilizara exclusivamente primitivas puras de React Native, componentes modales construidos a medida dentro del proyecto y utilidades estándar de Expo Vector Icons, asegurando compatibilidad cruzada entre dispositivos móviles y la web.

Otro punto de fricción crítico fue el manejo de la lógica temporal y los husos horarios. Dos de los casos de uso más importantes del sistema dependían de cálculos temporales precisos.
Inicialmente, las soluciones propuestas por la IA mezclaban fechas locales con formato UTC o utilizaban métodos de comparación de fechas de JavaScript que generaban inconsistencias sutiles según la zona horaria del cliente (diferencias entre UTC-3 y UTC). Esto provocaba que compras recién iniciadas expiraran antes de tiempo o que el contador de días calculara un día de más o de menos. La resolución no provino de pedirle a la IA que "adivinara" una corrección, sino de tomar una decisión de arquitectura explícita: definí que el backend normalizara todas las fechas y que cualquier validación de expiración se realizara mediante aritmética simple de milisegundos. Una vez fijada esa regla de negocio en las entidades de dominio, la IA pudo implementarla de manera sólida y consistente.

El problema más grande para mi fue la gestión de los límites de uso y las cuotas de las herramientas de IA. En medio de una sesión intensa de desarrollo, toparse de golpe con el aviso de plan agotado te corta el ritmo de trabajo por completo y te obliga a parar por horas. Esta limitación me obligó a ser mucho más estratégico y disciplinado: dejé de hacer consultas improvisadas y empecé a estructurar prompts mucho más claros y agrupados, dividiendo bien qué charlar con Claude para no gastar cupo en cosas menores y qué resolver directamente con Google Antigravity en el código. Además, aprendí a lidiar con el desgaste del contexto; cuando una conversación se hacía demasiado larga, el modelo empezaba a perder detalles del proyecto o a contradecirse, lo que me obligaba a reiniciar sesiones resumiendo con precisión el estado actual del desarrollo.

CONCLUSIONES FINALES

Haber completado este proyecto de punta a punta con asistencia de Inteligencia Artificial me dejó aprendizajes que van mucho más allá de las herramientas puntuales que usé. La conclusión más contundente es que la IA no vino a reemplazar el criterio del desarrollador, sino a exigirle que sea mejor ingeniero. Si uno encara un proyecto sin entender qué es una arquitectura limpia, cómo separar responsabilidades o cómo modelar un dominio complejo, la IA simplemente va a generar código desordenado y malas prácticas a una velocidad récord. Por el contrario, cuando se tienen claros los fundamentos técnicos, la herramienta se convierte en un acelerador impresionante que permite materializar ideas con un nivel de prolijidad que antes llevaba semanas alcanzar.

Otro gran aprendizaje fue revalorizar el rol de las pruebas automatizadas. Programar junto a una IA cambia por completo el ritmo de trabajo: se escribe y se refactoriza muy rápido. En ese contexto, la única forma de avanzar con tranquilidad sin romper nada en el camino es tener un arnés de seguridad sólido. Haber invertido tiempo en construir pruebas en Jest y las suites de Playwright fue la clave para no perder el control del proyecto. Cada vez que la IA sugería un cambio o agregaba una funcionalidad, correr los tests en segundos me confirmaba si íbamos por el camino correcto o si se había roto alguna funcionalidad.

En lo personal, este trabajo práctico me ayudó a mejorar cómo encarar los proyectos, tanto los académicos como los laborales. Pude dejar de gastar energía mental en tareas mecánicas, para concentrarme en lo que verdaderamente aporta valor: diseñar una solución robusta, cuidar la experiencia de los usuarios, prevenir problemas y asegurar que el negocio funcione como debe.

En definitiva, la experiencia me demostró que la inteligencia artificial no es magia ni un piloto automático; es un instrumento extraordinario en manos de quién sabe qué quiere construir y por qué. Saber combinar la visión estratégica con herramientas como Claude y Google Antigravity, manteniendo siempre el ojo crítico sobre el código, es sin duda la habilidad más valiosa que me llevo de esta cursada para mi futuro profesional.